

Guide for KuroTools, a set of tools for modding Kuro no Kiseki PC version

Guide written by [Twn](#), the tool itself written by Twn, [SoftBrilliant](#) and [hell259](#).

Introduction

As of the day of writing this guide, Kuro no Kiseki PC version from CLE has been out for a week and proper modding has not started (as far as we know) except from basic model swapping. This guide is here to present the features of KuroTools, a set of tools for modding this game. Note that the tools are still experimental and might not work well in some cases, so if you encounter a problem, please open an issue on the main repository <https://github.com/nnguyen259/KuroTools/issues>. Also feel free to drop by the [Kiseki Modding Discord](#) if you want to contribute or if you have trouble using the tools.

Finally, we encourage all modders who successfully made a mod or a showcase video of their mod to share it with us and the community!

Setup the environment

Requirements

First, as stated in the readme of the KuroTools github repository, Python version 10 and superior is required. You can find the install [here](#).

Also you should make sure you don't have other installations of Python that could bring trouble.

The chinese and korean files from CLE PC version are encrypted with Blowfish, for which a decryptor was written in pure python [here](#), so there is no package required for the decryption. However, the files are also compressed with Zstd, which doesn't exist in a working pure python form anywhere, so we are using the package *zstandard*. You can install zstandard using "pip install zstandard" in cmd on Windows. (Hopefully it is using the right pip for the version of Python that will be running KuroTools). In any case, if your files are not encrypted, the package shouldn't even be needed.

The tools

In this section we will describe the tools composing KuroTools and explain what they are used for. There are two tools, one is for TBL files editing (**`tbl2json.py`** and **`json2tbl.py`**) and the other is for DAT files editing (**`dat2py.py`**).

TBL files

TBL files are located within the game directory, inside `\tc\table` for Traditional Chinese and `\kr\table` for Korean. If you are using the existing English patch, the patch is likely replacing the ones inside the `tc` folder.

The tbl files contain various data ranging from NPC parameters, to item descriptions, character names, etc. A TBL file contains “headers”, which describe the size and count of each entry in the table (see a TBL file as a table), as well as the actual list of entries. For example, `t_name` will contain the data for all characters in the game, and `t_item` will be the same for items. If you want to add more items to the game, you will need to expand `t_item.tbl`.

The purpose of the `tbl2json.py` script is to transform the content of a TBL file:

	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F
0000:	23	54	42	4C	01	00	00	00	41	63	68	69	65	76	65	6D	#	T	B	L	.	.	.	.	.	.	.	.	.	.	.
0010:	65	6E	74	54	61	62	6C	65	44	61	74	61	00	00	00	00	e	n	t	T	a	b	l	e	D	a	t	a	.	.	.
0020:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.
0030:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.
0040:	00	00	00	00	00	00	00	00	99	89	FF	CC	58	00	00	00	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.
0050:	48	00	00	00	9F	00	00	00	01	00	00	00	00	00	00	00	H	.	.	.	.	.	.	.	.	.	.	.	.	.	.
0060:	32	00	00	00	02	00	00	00	10	2D	00	00	00	00	00	00	2	.	.	.	.	.	.	.	.	.	.	.	.	.	.
0070:	00	00	00	00	8C	28	00	00	28	00	00	00	01	00	00	00	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.
0080:	00	00	00	00	00	00	00	00	10	2D	00	00	00	00	00	00	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.
0090:	11	2D	00	00	00	00	00	00	45	2D	00	00	00	00	00	00	-	.	.	.	.	.	.	.	.	.	.	.	.	.	.
00A0:	01	00	00	00	01	00	00	00	32	00	00	00	03	00	00	00	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.
00B0:	B4	2D	00	00	00	00	00	00	00	00	00	00	00	00	04	00	-	.	.	.	.	.	.	.	.	.	.	.	.	.	.
00C0:	28	00	00	00	02	00	00	00	00	00	00	00	00	00	00	00	(	.	.	.	.	.	.	.	.	.	.	.	.	.	.
00D0:	B4	2D	00	00	00	00	00	00	B5	2D	00	00	00	00	00	00	-	.	.	.	.	.	.	.	.	.	.	.	.	.	.
00E0:	E9	2D	00	00	00	00	00	00	01	00	00	00	02	00	00	00	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.
00F0:	32	00	00	00	04	00	00	00	58	2E	00	00	00	00	00	00	2	.	.	.	.	.	.	.	.	.	.	.	.	.	.
0100:	00	00	00	00	00	00	00	00	28	00	00	00	04	00	00	00	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.
0110:	00	00	00	00	00	00	00	00	58	2E	00	00	00	00	00	00	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.
0120:	59	2E	00	00	00	00	00	00	8D	2E	00	00	00	00	00	00	Y	.	.	.	.	.	.	.	.	.	.	.	.	.	.
0130:	01	00	00	00	03	00	00	00	32	00	00	00	05	00	00	00	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.

into a human readable format (json) which looks like this:

```
{
  "headers": [
    {
      "name": "AchievementTableData",
      "schema": "FALCOM_PS4"
    }
  ],
  "data": [
    {
      "name": "AchievementTableData",
      "data": [
        {
          "achievement_category": 1,
          "achievement_id": 0,
          "achievement_objective_1": 50,
          "achievement_objective_param_1": 2,
          "arr1": [],
          "short1": 10380,
          "short2": 0,
          "int1": 40,
          "int2": 1,
          "long1": 0,
          "flag": "",
          "achievement_name": "엘라인먼트 룬(L)을 LV.2까지 올리자",
          "achievement_description": "L.G.C. 엘라인먼트 <C3>LAW (룬)</C>의 레벨을 <C3>2</C>까지 올린다.\n<c698>달성 보수</c>:"
        },
        {
          "achievement_category": 1,
          "achievement_id": 1,
          "achievement_objective_1": 50,
          "achievement_objective_param_1": 2,
          "arr1": [],
          "short1": 10380,
          "short2": 0,
          "int1": 40,
          "int2": 1,
          "long1": 0,
          "flag": "",
          "achievement_name": "엘라인먼트 룬(L)을 LV.2까지 올리자",
          "achievement_description": "L.G.C. 엘라인먼트 <C3>LAW (룬)</C>의 레벨을 <C3>2</C>까지 올린다.\n<c698>달성 보수</c>:"
        }
      ]
    }
  ]
}
```

Once each field is edited, or if new entries have been added, the file can then be recompiled into TBL format readable by the game using the script called `json2tbl.py`.

Usage

Decompiling a table

The usage of both `tbl2json` and `json2tbl` is extremely simple as there are no other arguments than the path of the file to be decompiled/recompiled. In fact, a simple drag and drop will be enough to create a decompiled json.

	.git	8/8/2022 5:09 PM	File folder	
	__pycache__	8/6/2022 2:30 PM	File folder	
	disasm	8/2/2022 4:40 PM	File folder	
	font	8/1/2022 12:49 AM	File folder	
	korean_new	8/8/2022 4:55 PM	File folder	
	korean_scripts	7/31/2022 9:56 AM	File folder	
	korean_tables	8/6/2022 3:36 PM	File folder	
	KuroTools	8/7/2022 3:49 PM	File folder	
	lib	8/8/2022 12:03 AM	File folder	
	schemas	8/7/2022 1:06 PM	File folder	
	scripts	8/8/2022 6:44 PM	File folder	
	.gitignore	7/31/2022 1:35 PM	GITIGNORE File	1 KB
	README.md	8/6/2022 4:03 PM	MD Document	4 KB
	dat2py.py	8/7/2022 8:55 PM	Python File	2 KB
	json2tbl.py	8/8/2022 4:10 PM	Python File	4 KB
	processcle.py	8/6/2022 1:26 PM	Python File	2 KB
	tbl2json.py	8/8/2022 4:12 PM	Python File	6 KB
	t_achievement.tbl	8/8/2022 6:04 PM	TBL File	34 KB

If for some reason there was a need to decompile the files using a command line, you can do so using:

```
<PYTHON_PATH> tbl2json.py <TBL_FILE_PATH>
```

Recompiling a table

As for the recompilation process, it is done the same way but with the script `json2tbl.py`:

.git	8/8/2022 5:09 PM	File folder	
__pycache__	8/6/2022 2:30 PM	File folder	
disasm	8/2/2022 4:40 PM	File folder	
font	8/1/2022 12:49 AM	File folder	
korean_new	8/8/2022 4:55 PM	File folder	
korean_scripts	7/31/2022 9:56 AM	File folder	
korean_tables	8/6/2022 3:36 PM	File folder	
KuroTools	8/7/2022 3:49 PM	File folder	
lib	8/8/2022 12:03 AM	File folder	
schemas	8/7/2022 1:06 PM	File folder	
scripts	8/8/2022 6:44 PM	File folder	
t_achievement.json	8/8/2022 6:45 PM	Adobe After Effect...	78 KB
.gitignore	7/31/2022 1:35 PM	GITIGNORE File	1 KB
README.md	8/6/2022 4:03 PM	MD Document	4 KB
dat2py.py	8/7/2022 8:55 PM	Python File	2 KB
json2tbl.py	8/8/2022 4:10 PM	Python File	4 KB
processcle.py	8/6/2022 1:26 PM	Python File	2 KB
tbl2json.py	8/8/2022 4:12 PM	Python File	6 KB

Or in command line:

<PYTHON_PATH> json2tbl.py <JSON_FILE_PATH>

File structure

The JSON decompiled file for t_achievement.tbl is shown below:

```
{
  "headers": [
    {
      "name": "AchievementTableData",
      "schema": "FALCOM_PS4"
    }
  ],
  "data": [
    {
      "name": "AchievementTableData",
      "data": [
        {
          "achievement_category": 1,
          "achievement_id": 0,
          "achievement_objective_1": 50,
          "achievement_objective_param_1": 2,
          "arr1": [],
          "short1": 10380,
          "short2": 0,
          "int1": 40,
          "int2": 1,
          "long1": 0,
          "flag": "",
          "achievement_name": "엘라인먼트 룬(L)을 LV.2까지 올리자",
          "achievement_description": "L.G.C. 엘라인먼트 <C3>LAW (룬)</C>의 레벨을 <C3>2</C>까지 올린다.\n<c698>달성 보수</c>:"
        },
        {
          "achievement_category": 1,
          "achievement_id": 1,
          "achievement_objective_1": 50,
          "achievement_objective_param_1": 3,
          "arr1": [],
          "short1": 0,
          "short2": 4,
          "int1": 40,
          "int2": 2,
          "long1": 0,
          "flag": "",
          "achievement_name": "엘라인먼트 룬(L)을 LV.3까지 올리자",
          "achievement_description": "L.G.C. 엘라인먼트 <C3>LAW (룬)</C>의 레벨을 <C3>3</C>까지 올린다.\n<c698>달성 보수</c>:"
        },
        {
          "achievement_category": 1,
          "achievement_id": 2,
          "achievement_objective_1": 50,
          "achievement_objective_param_1": 3,
          "arr1": [],
          "short1": 0,
          "short2": 4,
          "int1": 40,
          "int2": 2,
          "long1": 0,
          "flag": "",
          "achievement_name": "엘라인먼트 룬(L)을 LV.3까지 올리자",
          "achievement_description": "L.G.C. 엘라인먼트 <C3>LAW (룬)</C>의 레벨을 <C3>3</C>까지 올린다.\n<c698>달성 보수</c>:"
        }
      ]
    }
  ]
}
```


You can edit it in any text editor, but there are better tools out there to edit json without risking to damage the format, for example [VSCode](#). If you chose a simple text editor though, know that there are a few rules to respect to make a valid json. First, respect the brackets {} hierarchy. Also after each entry a comma “,” should follow, except for the last one. This can be the source of errors when recompiling the json and you could think the python script is the culprit, when really it’s just due to a bad json formatting.

The json starts with the “headers” list of headers names and schemas. The schema name (here “FALCOM_PS4”), corresponds to a certain version of the AchievementTableData header. It has been implemented like that because multiple versions of AchievementTableData can exist, since they can be improved between games/ports by the official developers. For example CLE port adds new data to the *t_quest* table, this is why there is a “CLE_PC” version of the QuestTitle header, and also a “FALCOM_PS4” one.

The schemas are containing infos about each field of an entry in the table (a name to understand the nature of the field, example is “achievement_name”, and the size of the element (necessary to recompile data)).

DAT files

The DAT files are located in *\tc\lscript* for Traditional Chinese and *\kr\lscript* for Korean. Those files contain a list of functions/events used in different contexts. For example, all scripted scenes in the game are contained within the script/scena folder, while animation/crafts events are scripted inside script/ani. You can technically write your own functions for crafts by editing the ani files of the character of your choice.

Usage

The tool used to decompile the script files is a script called dat2py.py. This script will take a .dat file and produce a .py file that contains all the data in a more readable form.

Decompiling a script file

To use this tool, just drag and drop your dat file onto dat2py.py like this:

Name	Date modified	Type	Size
.git	8/8/2022 5:09 PM	File folder	
__pycache__	8/6/2022 2:30 PM	File folder	
disasm	8/2/2022 4:40 PM	File folder	
font	8/1/2022 12:49 AM	File folder	
korean_new	8/8/2022 4:55 PM	File folder	
korean_scripts	7/31/2022 9:56 AM	File folder	
korean_tables	8/6/2022 3:36 PM	File folder	
KuroTools	8/7/2022 3:49 PM	File folder	
lib	8/8/2022 12:03 AM	File folder	
schemas	8/7/2022 1:06 PM	File folder	
scripts	8/8/2022 6:44 PM	File folder	
c0410.dat	7/30/2022 3:47 PM	DAT File	173 KB
.gitignore	7/31/2022 1:35 PM	GITIGNORE File	1 KB
README.md	8/6/2022 4:03 PM	MD Document	4 KB
dat2py.py	8/7/2022 8:55 PM	Python File	2 KB
json2tbl.py	8/8/2022 4:10 PM	Python File	4 KB
processcscle.py	8/6/2022 1:26 PM	Python File	2 KB
tbl2json.py	8/8/2022 4:12 PM	Python File	6 KB

Again, if for some reason you need to use a command line, you can use this:

```
<PYTHON_PATH> dat2py.py <Path to the dat file to be disassembled> --markers
<True/False> --decompile <True/False>
```

Notice that this script offers two additional options, which are “markers” and “decompile”. By default, markers is set to False and decompile is set to True (what is happening if you just drag and drop your dat file on dat2py.py).

If markers was set to True, it would add additional instructions that are present in the dat file, which were most likely added by Falcom for debugging (to mark where each block of instructions start to help locate the bugs, most likely). It doesn’t actually do anything in game, so it is actually better to set it to False unless you want to check the integrity of your data.

As for the decompile option, if set to False, the script will radically change. The script will look more like the Assembly language, and a hidden stack (see [https://en.wikipedia.org/wiki/Stack_\(abstract_data_type\)](https://en.wikipedia.org/wiki/Stack_(abstract_data_type))) will evolve during the script. You will have to make sure you don’t break it otherwise crashes will happen. However, if you’re just changing the text (for example, for a translation patch), or simple values in the script, this format is probably the safest one.

Decompile set to True will attempt to merge instructions into function calls to improve readability and make modding more accessible to non programmers (as much as possible). For example, a decompiled script block of code will look like this:

```

CallFunction("wait_prompt", [])
CallFunction("chr_look_dir", [FLOAT(0.2999999523162842), FLOAT(0.0), FLOAT(0.0), INT(1)])
CallFunction("chr_look_chr", [FLOAT(0.2999999523162842), INT(0), INT(1)])
Command("SetAnimation", ["AniEvTemune", INT(2), INT(1)])
Command("Cmd_text_06", ["Right, let's get out of here, then.", "<#E_8#M_0#B_0>", INT(1)])
CallFunction("wait_prompt", [])
CallFunction("sound_stop_bgm", [INT(0), INT(3)])
CallFunctionFromAnotherScript("sound", "StopEnvSe", [INT(1)])
CallFunction("fade_out", [INT(0), FLOAT(1.0), INT(0), FLOAT(1.0)])
CallFunction("fade_wait", [INT(0)])
CallFunction("wait", [INT(1)])
CallFunction("EVENT_FINALIZE", [])
AssignVar("VAR_0", UNDEF(0x0))
SAVERESULT(0)
Return()

```

While the same block of instructions in decompile = False mode, will give this:

```

PUSHSTRING("AniEvTemune")
PUSHINTEGER(2)
PUSHINTEGER(1)
RUNCMD(3, "SetAnimation")
POP(12)
PUSHSTRING("Right, let's get out of here, then.")
PUSHSTRING("<#E_8#M_0#B_0>")
PUSHINTEGER(1)
RUNCMD(3, "Cmd_text_06")
POP(12)
#Calling wait_prompt
PUSHCALLERFUNCTIONINDEX()
PUSHRETURNADDRESS("Loc_1649")
CALL("wait_prompt")

Label("Loc_1649")

#Calling sound_stop_bgm
PUSHCALLERFUNCTIONINDEX()
PUSHRETURNADDRESS("Loc_1650")
PUSHINTEGER(0)
PUSHINTEGER(3)
CALL("sound_stop_bgm")

```

In the decompile = True mode, a simple function call can be written in a single line, and all of the PUSHCALLERFUNCTIONINDEX, PUSHRETURNADDRESS, are done automatically. A simple function call encapsulates all the overhead and makes the script much more readable. That is why decompile = True is the default mode for modders that might not know the concept of stack. However, the decompilation itself is not perfect, and some stack operations could randomly appear (like POP).

Recompiling a script file

This operation is even simpler than before, as you only need to run the output of the decompilation step (the script .py file) with Python. It comes back to simply double clicking the output script like this:

.git	8/8/2022 8:56 PM	File folder	
__pycache__	8/6/2022 2:30 PM	File folder	
disasm	8/2/2022 4:40 PM	File folder	
font	8/1/2022 12:49 AM	File folder	
korean_new	8/8/2022 4:55 PM	File folder	
korean_scripts	7/31/2022 9:56 AM	File folder	
korean_tables	8/6/2022 3:36 PM	File folder	
KuroTools	8/7/2022 3:49 PM	File folder	
lib	8/8/2022 12:03 AM	File folder	
schemas	8/7/2022 1:06 PM	File folder	
scripts	8/8/2022 6:44 PM	File folder	
.gitignore	7/31/2022 1:35 PM	GITIGNORE File	1 KB
README.md	8/6/2022 4:03 PM	MD Document	4 KB
c0410.py	8/8/2022 9:03 PM	Python File	520 KB
dat2py.py	8/8/2022 8:55 PM	Python File	2 KB
json2tbl.py	8/8/2022 4:10 PM	Python File	4 KB
processcle.py	8/6/2022 1:26 PM	Python File	2 KB
tbl2json.py	8/8/2022 4:12 PM	Python File	6 KB

Again, if for some reason you need to use a command line, you'd just enter this:

<PYTHON_PATH> <Path to the py file obtained after the decompilation step>

File structure

The structure of the decompiled script files is a bit more complex than the tbl one:

First you have the list of all functions used in the script file, identified by their name (which must be unique):

```

add_function(
name= "CHR_DISABLE",
input_args = [UNDEF(0x1)],
output_args = [],
b0= 0x1,
b1= 0x0,
)

add_function(
name= "CHR_ENABLE",
input_args = [UNDEF(0x1)],
output_args = [],
b0= 0x1,
b1= 0x0,
)

add_function(
name= "CHR_HEAD_NO",
input_args = [UNDEF(0x1)]

```

Each function has input arguments that in the decompiled version of the script are named "PARAM_0", "PARAM_1", "PARAM_2", ...

B0 and B1 are data that are still not clear as of now.

After the long list of functions, you will find the content of the first function in the script:

```

add_function(
    name= "wait_prompt",
    input_args = [],
    output_args = [],
    b0= 0x1,
    b1= 0x0,
)

#-----
#original file addr: 0x13c1c
set_current_function("flag")
add_struct(
    id = -1,
    nb_sthl = 0x3,
    array2 = [INT(0), UNDEF(0x0), INT(0), UNDEF(0x0), UNDEF(0x0), UNDEF(0x2)],
)

#Instructions flag

Command("Cmd_system_00", [LoadVar("PARAM_0")])
AssignVar("VAR_1", LoadResult(0))
SAVERESULT(0)
Return()
#-----

```

The first function in this particular script is named *flag* as indicated by the “set_current_function(“flag”)” call. This is how you tell the script that your function *flag* starts there.

Immediately following the set_current_function, you will find the “add_struct” part.

This section can be very long, and list a few data structures that are likely used as default values for all commands and function calls that the function itself contains. You don’t have to worry about them as they are largely unused, since all arguments are present in all function calls you will make/Falcom makes. They would probably only be used if for some reason you call a function without specifying some of its parameters (but I never actually tried).

Finally the most important part of the function is the “Instructions” part.

Here in the “flag” function, it only consists of one command call to “Cmd_system_00”, that uses the first input argument of the function. This means somewhere in the script file, the function “flag” will be called with an input parameter, for example here:

```

CallFunction("map_event_box_set_enable", [INT(0), "ST_QS101_04_00"]
CallFunction("map_event_box_set_enable", [INT(0), "ST_QS101_04_00"]
CallFunction("map_event_box_set_enable", [INT(0), "ST_QS101_04_00"]
CallFunction("map_event_box_set_enable", [INT(0), "ST_QS101_04_00"]
CallFunction("map_event_box_set_enable", [INT(0), "ST_QS101_04_00"]
CallFunction("flag", [INT(300)])
JumpWhenTrue("Loc_85", LoadResult(0))
CallFunction("flag", [INT(10429)])

```

The first parameter, 300, is identified as PARAM_0 in the flag function instructions part.

Finally, the block:

```

AssignVar("VAR_1", LoadResult(0))
SAVERESULT(0)
Return()

```

is a very common pattern, that just ends the function by returning the result of the command CMD_System_00 (as indicated by the assignment of LoadResult(0) to VAR1, which is then SAVED as result before returning to the caller.

While all of this is very technical, I encourage that you keep reading at least the guide for the first mod at the end of this document where things should appear clearer, since you technically only need to know the right functions to add characters to the party or crafts.

Note on broken files

There are a few files that might look broken, or have inaccessible parts of code that will make the tool say :

```
WARNING: Likely a jump right after a return. Just ignore this unless you know what you're doing. Output file should be fine.
```

In general this is no cause of concern. This is due to this particular pattern:

```
2606      CALL(401)
2607
2608      Label("Loc_1072")
2609
2610      LOADRESULT(0)
2611      SAVERESULT(0)
2612      POP(12)
2613      EXIT()
2614      JUMP("Loc_81")
2615
2616      Label("Loc_80")
2617
2618      #Calling party_get_chrid
2619      PUSHCALLERFUNCTIONINDEX(0x54)
2620      PUSHRETURNADDRESS("Loc_1073")
2621      PUSHINTEGER(-1)
2622      PUSHINTEGER(1)
2623      CALL(401)
2624
2625      Label("Loc_1073")
2626
2627      LOADRESULT(0)
2628      RETRIEVEELEMENTATINDEX(-12)
2629      NONEQUAL()
2630      #Calling party_get_chrid
2631      PUSHCALLERFUNCTIONINDEX(0x54)
2632      PUSHRETURNADDRESS("Loc_1074")
2633      PUSHINTEGER(-1)
2634      PUSHINTEGER(1)
2635      CALL(401)
2636
```

result - 5 hits

Search "Loc_81" (5 hits in 1 file)

Line 12614:	JUMP("Loc_81")
Line 12661:	JUMP("Loc_81")
Line 12708:	JUMP("Loc_81")
Line 12755:	JUMP("Loc_81")
Line 12773:	Label("Loc_81")

You will notice that right after the EXIT() instruction line 2613, follows a jump. This doesn't make sense as the function exits right before the jump, so the JUMP will never be reached. This causes a problem with the stack calculation of the tool so a warning is displayed, but since the code is never executed anyway, nothing bad will happen.

Also note that mon9996_c00.dat in ani folder is broken. The function headers in this file are 16 bytes long instead of the usual 32 bytes, which causes the tool to hang when decompiling it.

Guides

How to add schemas to the TBL tool

This section will be done later (likely by SoftBrilliant)

Our first mod (TBL & Script editing)

We will now create our first mod, the end goal being shown in this video:

<https://www.youtube.com/watch?v=xTLBBp6cc2s>

First, find the script corresponding to the map where you want your events to happen. I'll pick C0000 because I want to mod the very beginning of the game, just outside the Arkride Solution Office.

The goal of this mod will be to add a dummy NPC in the middle of the street that displays a multiple choices menu. There will be two choices:

- Add a character to the party
- Equip this character with items

The first step is to decompile C0000.dat into a .py file in order to edit it. To do that follow the steps described in the tools section.

Once this step is done and you have your .py file, we will first want to locate the "Init" function of the map. This function is bound to be executed as the exe specifically looks for it after loading the script. We will want to add our new dummy NPC through this function along with the rest of the NPCs.

To locate it in the .py file, just press CTRL + F and search for "Instructions Init" to find the content of this function.

```
#Instructions Init

CallFunction("map_load_npc_table", ["t_npc_c0000"])
CallFunction("map_load_mons_table", ["t_mon_c0000"])
AssignVar("VAR_0", UNDEF(0x0))
SAVERESULT(0)
Return()
```

Here we see that the Init function seems to do only two things: load the NPC table (which is a tbl, "t_npc_c0000" as indicated in the second parameter of "CallFunction", and load the monster table (also a tbl).

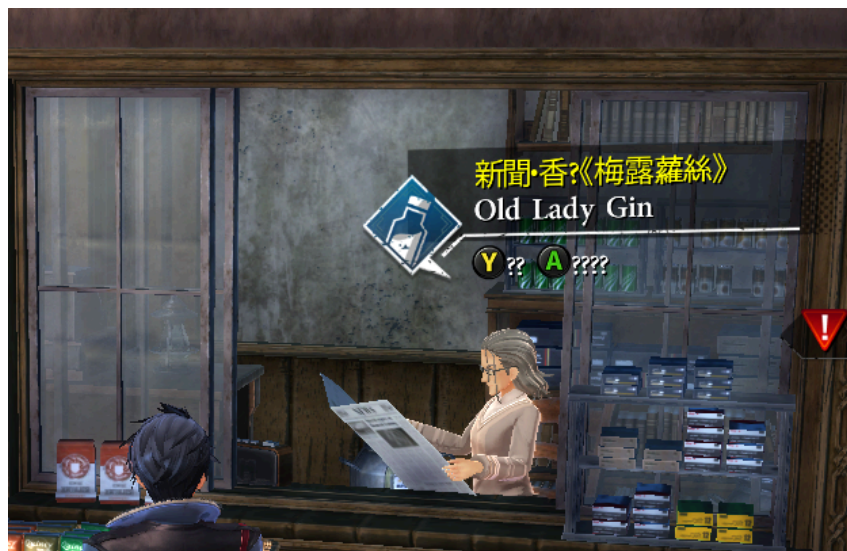
There might be ways to add a NPC without relying on those tables but we will play along and try to add our NPC to the t_npc_c0000.tbl.

Adding a new NPC on C0000

The decompiled t_npc_c0000 entries look like this:

```
{
  "char_id": 11057,
  "start_scena_flag": [
    10013
  ],
  "int1": 0,
  "end_scena_flag": [
    10845
  ],
  "int2": 0,
  "map": "c0000",
  "x": 4.757999897003174,
  "y": 0.5,
  "z": 18.2549991607666,
  "orientation_angle": 210.39999389648438,
  "range_interaction": 1.5,
  "int": 41101,
  "flt7": 0.0,
  "flt8": 0.0,
  "flt9": 0.0,
  "flt10": 0.0,
  "text1": "AGR",
  "talk_setting_function": "c0000.TK_setting_gin",
  "first_animation_function": "npc_setting.AniSitNewspaper",
  "int3": 0,
  "int4": 0,
  "talk_function": "c0000.TK_GIN",
  "text2": "",
  "int5": 0,
  "int6": 0,
  "int7": 20,
  "int8": 0
},
```

This one entry in particular refers to this NPC:



We know that, first because of the “char_id” which equals 11057. If you search for a character with id 11057 in the decompiled t_name table, you will find:

```
{
  "character_id": 11057,
  "name": "Old Lady Gin",
  "texture": "chr5400",
  "face": "chrf540_face",
  "model": "chrx001",
  "long1": 30,
  "text1": "F",
  "text2": "npc",
  "long2": 0,
  "text3": "",
  "text4": ""
},
```

Also, you will notice that it uses C0000.TK_GIN as its talk function (the function triggered when interacting with it with the left click). However this NPC has multiple versions of itself in

t_npc_c0000, depending on the progress in story, which is described by the scenario flags. Here I'm not 100% sure, but that NPC has 10013 as a start_scena_flag and 10845 as a end_scena_flag, which would mean those particular settings (Position on the map X Y Z, Default animation,...) are used when the current story flag value is inside the range [10013 - 10845].

Now, what would we do to add a new NPC on the map? Basically copying the whole entry of that old lady down in the bottom of the t_npc_c0000.json file. Let's also change the character identity itself to Shizuna, for fun, by changing chr_id from 11057 to 12 (Shizuna ID as found in t_name.tbl). We also keep the same scena flags so we ensure she will be visible at the beginning of the game. The only things we need to change are the position of Shizuna on the map, her default animation, and her talk function.

For the position on the map, I don't have better means than trial and error, specifying coordinates until you find a suitable place. Keep in mind that the Y coordinate is the altitude of the character.

For example, here I added an entry at the very end of the file. char_id is 12 for Shizuna, X coordinate is set to 0 (was previously 4 for the old lady). Everything else is unchanged.

```

707         "float10": 0.0,
708         "text1": "APY",
709         "talk_setting_function": "",
710         "first_animation_function": "npc_setting.AniEvHoldMugSit",
711         "int3": 0,
712         "int4": 0,
713         "talk_function": "",
714         "text2": "",
715         "int5": 0,
716         "int6": 0,
717         "int7": 0,
718         "int8": 0
719     },
720     {
721         "char_id": 12,
722         "start_scena_flag": [
723             10013
724         ],
725         "int1": 0,
726         "end_scena_flag": [
727             10845
728         ],
729         "int2": 0,
730         "map": "c0000",
731         "X": 0,
732         "Y": 0.5,
733         "Z": 18.2549991607666,
734         "orientation_angle": 210.39999389648438,
735         "range_interaction": 1.5,
736         "int": 41101,
737         "flt7": 0.0,
738         "flt8": 0.0,
739         "flt9": 0.0,
740         "flt10": 0.0,
741         "text1": "AGR",
742         "talk_setting_function": "c0000.TK_setting_gin",
743         "first_animation_function": "npc_setting.AniSitNewspaper",
744         "int3": 0,
745         "int4": 0,
746         "talk_function": "c0000.TK_GIN",
747         "text2": "",
748         "int5": 0,
749         "int6": 0,
750         "int7": 20,
751         "int8": 0
752     }
753 ]
754 }
755 ]
756 }

```

in game, this is what happens:



Setting the X coordinate to 0 translated Shizuna toward the street, but her altitude is still off (likely because the old lady is sitting on a chair or something.)

Also, talking to her will trigger the exact same dialog than when talking to the old lady. This is because we didn't change her functions in `t_npc_c0000.tbl`.

Finally, she doesn't have a sitting animation and is just standing, which could mean she is missing that specific newspaper animation (which makes sense) and reverted to a default one.

So first let's set her Y coordinate to 0. Let's also put blanks in all functions like this:

```

{
  "char_id": 12,
  "start_scena_flag": [
    10013
  ],
  "int1": 0,
  "end_scena_flag": [
    10845
  ],
  "int2": 0,
  "map": "c0000",
  "X": 0,
  "Y": 0,
  "Z": 18.2549991607666,
  "orientation_angle": 210.39999389648438,
  "range_interaction": 1.5,
  "int": 41101,
  "flt7": 0.0,
  "flt8": 0.0,
  "flt9": 0.0,
  "flt10": 0.0,
  "text1": "AGR",
  "talk_setting_function": "",
  "first_animation_function": "",
  "int3": 0,
  "int4": 0,
  "talk_function": "",
  "text2": "",
  "int5": 0,
  "int6": 0,
  "int7": 20,
  "int8": 0
}

```

Here's what happens:



As you can see, we can't interact with her at all this time. She also has the same animation than before, which will be fine for our first mod as we only need her to display a menu to select choices. So now, we only need to write a function that displays the choices we want for our mod.

Writing a new function to C0000: *TK_SHIZUNA*

NOTE: The following part was written with an old version of the tool; if you are using the latest one, all the orders of the parameters in the function calls are reversed (the list within the [] in each function call).

For that, let's get back to C0000.py. We are going to create a very simple function called TK_SHIZUNA (Verify that it doesn't exist before!).

To do that, we need to first add the function at the beginning of the py file. You can add it wherever you want in the list:

```
add_function(  
    name= "LP_EV_06_18_00",  
    input_args = [],  
    output_args = [],  
    b0= 0x0,  
    b1= 0x0,  
)  
add_function(  
    name= "TK_SHIZUNA",  
    input_args = [],  
    output_args = [],  
    b0= 0x0,  
    b1= 0x0,  
)  
add_function(  
    name= "LP_Garage",  
    input_args = [],  
    output_args = [],  
    b0= 0x0,  
    b1= 0x0,
```

In order to do that, just copy one of the existing add_function and adapt it a little: give it a name, here TK_SHIZUNA, that must be unique. Also, remove any input and output args because we won't need them. finally b0 and b1 can be set to 0 (I actually don't know what they do).

Now we need to add the instructions of the function itself. Let's add them at the end of the c0000.py file, right before the compile():

```
#####  
#original file addr: 0x3c0446  
set_current_function("YobikomiV_EDI")  
add_struct(  
    id = 933,  
    nb_sth1 = 0x0,  
    array2 = [INT(11079), UNDEF(0x0), INT(8780), UNDEF(0x0), INT(-4), UNDEF(0x0)],  
)  
  
#Instructions YobikomiV_EDI  
  
CallFunction("sound_play_voice", [INT(0), INT(-1), FLOAT(1.0), FLOAT(0.0), INT(-4), INT(8780), INT(11079)])  
AssignVar("VAR_0", UNDEF(0x0))  
SAVERESULT(0)  
Return()  
  
compile()
```

Where YobikomiV_EDI is the last function of the script file.


```
set_current_function("TK_SHIZUNA")
```

```
AssignVar("VAR_0", UNDEF(0x0))  
SAVERESULT(0)  
Return()
```

This above is the minimum code you need to have a working function. Notice that the set_current_function has the name of the function. The instructions themselves (The AssignVar and SAVERESULT) are simply meaning that we return “0”, in other words we signal to the game that everything went OK.

We also need to add our custom function to Shizuna in the json, like this:

```
{  
  "char_id": 12,  
  "start_scena_flag": [  
    10013  
  ],  
  "int1": 0,  
  "end_scena_flag": [  
    10845  
  ],  
  "int2": 0,  
  "map": "c0000",  
  "X": 0,  
  "Y": 0,  
  "Z": 18.2549991607666,  
  "orientation_angle": 210.39999389648438,  
  "range_interaction": 1.5,  
  "int": 0,  
  "flt7": 0.0,  
  "flt8": 0.0,  
  "flt9": 0.0,  
  "flt10": 0.0,  
  "text1": "PRY",  
  "talk_setting_function": "",  
  "first_animation_function": "npc_setting.AniEvSquatLoop",  
  "int3": 0,  
  "int4": 0,  
  "talk_function": "c0000.TK_SHIZUNA",  
  "text2": "",  
  "int5": 0,  
  "int6": 0,  
  "int7": 0,  
  "int8": 0  
}
```

Note that in the above entry, I modified a few fields from the GIN entry we used earlier, in order to remove the “Shop” icon.

In game, we get this:



Her unusual position is due to having set "first_animation_function" to
npc_setting.AniEvSquatLoop in t_npc_c0000.tbl

It becomes possible to interact with her but when left clicking, TK_SHIZUNA being empty, nothing happens. We now need to create our custom menu. For that we need to find functions/commands that can create such a menu.

Create a two choices menu for Shizuna

Below I'll provide a template that can accomplish a simple two choices menu, feel free to start from here:

Template for a two choices menu
<pre>set_current_function("TK_SHIZUNA") CallFunction("TALK_BEGIN", [FLOAT(4.0), INT(0)]) CallFunction("menu_create", [INT(28), INT(0), INT(1), INT(1)]) CallFunction("menu_additem", [INT(0), "Add a character to the party", INT(1)]) CallFunction("menu_additem", [INT(1), "Add an item to the inventory", INT(1)]) CallFunction("menu_open", [INT(1), INT(-1), INT(-1), INT(1)]) CallFunction("menu_wait", [INT(1)]) AssignVar("VAR_0", LoadResult(0)) CallFunction("menu_close", [INT(1)]) CallFunction("menu_is_canceled", [INT(1)]) JumpWhenFalse("Loc_MenuCanceled", isfalse(LoadResult(0))) SAVERESULT(0) JumpWhenFalse("Loc_Choice0", equal(LoadResult(0), INT(0))) JumpWhenFalse("Loc_Choice1", equal(LoadResult(0), INT(1))) JUMP("Loc_ChoiceMade") Label("Loc_Choice0") Command("Cmd_text_06", ["Choice 0", INT(65534)]) CallFunction("wait_prompt", []) JUMP("Loc_ChoiceMade") Label("Loc_Choice1") Command("Cmd_text_06", ["Choice 1", INT(65534)]) CallFunction("wait_prompt", []) JUMP("Loc_ChoiceMade") Label("Loc_MenuCanceled") Command("Cmd_text_06", ["Menu Canceled", INT(65534)]) CallFunction("wait_prompt", []) Label("Loc_ChoiceMade") CallFunction("TALK_END", [])</pre>

```
AssignVar("VAR_0", UNDEF(0x0))
SAVERESULT(0)
Return()
```

Without entering into the details, what matters in that code is listed below:

- the bold area, which specifies the location in code handling each of the choice. Notice that INT(0) and INT(1), last parameters of JumpWhenFalse, are the identifiers of each choice.
- The red area is what happens when you select the first choice. It will display "Choice0"
- The green area is what happens when you select the second choice. It will display "Choice1"
- The blue area is what happens when you right click the menu (canceling).

From there, feel free to add more choices by adding Locations and JumpWhenFalses like the first two ones.

We now need to implement the "add character to party" and "add item to inventory" behaviors. For this, we just need to find the right functions, which will surely have been implemented somewhere in the game already.

Adding a character to the party

In fact, adding a character to the party is as simple as using a single function call, which is given below:

```
CallFunction("party_join", [INT(0), INT(-1), INT(12)])
```

The INT(12) here is Shizuna's ID. As for the two first parameters, I actually don't know what is their purpose.

You can then rewrite the "Choice0" area of the code like this:

```
Label("Loc_Choice0")

Command("Cmd_text_06", ["Adding Shizuna to the party", INT(65534)])
CallFunction("wait_prompt", [])
CallFunction("party_join", [INT(0), INT(-1), INT(12)])
JUMP("Loc_ChoiceMade")
```

Equip the character

As for the second choice, we will borrow the code from the function "SetEquipELAINEp2" from system.dat. Just decompile the dat and look for the instructions for this function, which are given below:


```
#Instructions SetEquipELAINEep2
```

```
CallFunction("chr_status_set", [INT(25), INT(0), INT(11)])
CallFunction("chr_status_clear_craft", [INT(11)])
CallFunction("chr_status_add_craft", [INT(0), INT(11)])
CallFunction("chr_status_set_equip", [INT(1), INT(1), INT(601), INT(11)])
CallFunction("chr_status_set_equip", [INT(1), INT(2), INT(704), INT(11)])
CallFunction("chr_status_set_equip", [INT(1), INT(3), INT(1005), INT(11)])
CallFunction("chr_status_set_equip", [INT(1), INT(4), INT(1205), INT(11)])
CallFunction("chr_status_set_equip", [INT(1), INT(5), INT(1631), INT(11)])
CallFunction("chr_status_set_equip", [INT(1), INT(6), INT(1634), INT(11)])
CallFunction("chr_status_set_hollowcore", [INT(1), INT(3348), INT(11)])
CallFunction("item_set_hollow_core_level", [INT(0), INT(2), INT(3348)])
CallFunction("chr_status_set_artsdriver", [INT(1), INT(3450), INT(11)])
CallFunction("chr_status_set_quartz_slot_level", [INT(0), INT(1), INT(0), INT(11)])
CallFunction("chr_status_set_quartz_slot_level", [INT(0), INT(1), INT(1), INT(11)])
CallFunction("chr_status_set_quartz_slot_level", [INT(0), INT(1), INT(2), INT(11)])
CallFunction("chr_status_set_quartz_slot_level", [INT(0), INT(1), INT(4), INT(11)])
CallFunction("chr_status_set_quartz_slot_level", [INT(0), INT(1), INT(5), INT(11)])
CallFunction("chr_status_set_quartz_slot_level", [INT(0), INT(1), INT(6), INT(11)])
CallFunction("chr_status_set_quartz_slot_level", [INT(0), INT(1), INT(8), INT(11)])
CallFunction("chr_status_set_quartz_slot_level", [INT(0), INT(1), INT(9), INT(11)])
CallFunction("chr_status_set_quartz_slot_level", [INT(0), INT(1), INT(10), INT(11)])
CallFunction("chr_status_set_quartz_slot_level", [INT(0), INT(1), INT(12), INT(11)])
CallFunction("chr_status_set_quartz_slot_level", [INT(0), INT(1), INT(13), INT(11)])
CallFunction("chr_status_set_quartz_slot_level", [INT(0), INT(1), INT(14), INT(11)])
CallFunction("chr_status_set_quartz", [INT(1), INT(0), INT(3901), INT(11)])
CallFunction("chr_status_set_quartz", [INT(1), INT(1), INT(4320), INT(11)])
CallFunction("chr_status_set_quartz", [INT(1), INT(2), INT(4020), INT(11)])
CallFunction("chr_status_set_quartz", [INT(1), INT(4), INT(3911), INT(11)])
CallFunction("chr_status_set_quartz", [INT(1), INT(5), INT(4011), INT(11)])
CallFunction("chr_status_set_quartz", [INT(1), INT(6), INT(4000), INT(11)])
CallFunction("chr_status_set_quartz", [INT(1), INT(8), INT(4101), INT(11)])
CallFunction("chr_status_set_quartz", [INT(1), INT(9), INT(3701), INT(11)])
CallFunction("chr_status_set_quartz", [INT(1), INT(10), INT(4310), INT(11)])
CallFunction("chr_status_set_quartz", [INT(1), INT(12), INT(4220), INT(11)])
CallFunction("chr_status_set_quartz", [INT(1), INT(13), INT(4211), INT(11)])
CallFunction("chr_status_set_quartz", [INT(1), INT(14), INT(4120), INT(11)])
CallFunction("item_set_artsdriver_slot_all_unlock", [INT(3450)])
CallFunction("item_set_arts_plugin", [INT(1), INT(3543), INT(0), INT(3450)])
```

Note that since Elaine's ID is 11 and we don't have Elaine in our party, we will need to change all INT(11) with INT(12) to equip Shizuna. So move everything from there to TK_SHIZUNA in the Choice1 location, with their ID replaced. Also note that not every function called here exists in C0000. This means you need to call them from system.dat, which is done like this:

```

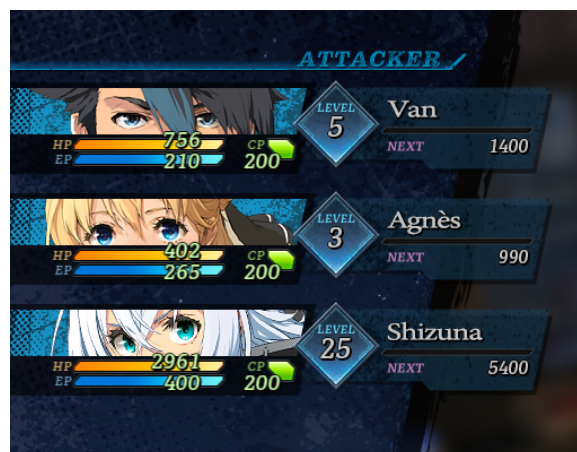
Label("Loc_Choice1")

Command("Cmd_text_06", ["Equiping stuff", INT(65534)])
CallFunction("wait_prompt", [])
CallFunctionFromAnotherScript("system", "chr_status_set", [INT(25), INT(0), INT(12)])
CallFunctionFromAnotherScript("system", "chr_status_clear_craft", [INT(12)])
CallFunctionFromAnotherScript("system", "chr_status_add_craft", [INT(0), INT(12)])
CallFunctionFromAnotherScript("system", "chr_status_set_equip", [INT(1), INT(1), INT(601), INT(12)])
CallFunctionFromAnotherScript("system", "chr_status_set_equip", [INT(1), INT(2), INT(704), INT(12)])
CallFunctionFromAnotherScript("system", "chr_status_set_equip", [INT(1), INT(3), INT(1005), INT(12)])
CallFunctionFromAnotherScript("system", "chr_status_set_equip", [INT(1), INT(4), INT(1205), INT(12)])
CallFunctionFromAnotherScript("system", "chr_status_set_equip", [INT(1), INT(5), INT(1631), INT(12)])
CallFunctionFromAnotherScript("system", "chr_status_set_equip", [INT(1), INT(6), INT(1634), INT(12)])
CallFunctionFromAnotherScript("system", "chr_status_set_hollowcore", [INT(1), INT(3348), INT(12)])
CallFunctionFromAnotherScript("system", "item_set_hollow_core_level", [INT(0), INT(2), INT(3348)])
CallFunctionFromAnotherScript("system", "chr_status_set_artsdriver", [INT(1), INT(3450), INT(12)])
CallFunctionFromAnotherScript("system", "chr_status_set_quartz_slot_level", [INT(0), INT(1), INT(0), INT(12)])
CallFunctionFromAnotherScript("system", "chr_status_set_quartz_slot_level", [INT(0), INT(1), INT(1), INT(12)])
CallFunctionFromAnotherScript("system", "chr_status_set_quartz_slot_level", [INT(0), INT(1), INT(2), INT(12)])
CallFunctionFromAnotherScript("system", "chr_status_set_quartz_slot_level", [INT(0), INT(1), INT(4), INT(12)])
CallFunctionFromAnotherScript("system", "chr_status_set_quartz_slot_level", [INT(0), INT(1), INT(5), INT(12)])
CallFunctionFromAnotherScript("system", "chr_status_set_quartz_slot_level", [INT(0), INT(1), INT(6), INT(12)])
CallFunctionFromAnotherScript("system", "chr_status_set_quartz_slot_level", [INT(0), INT(1), INT(8), INT(12)])
CallFunctionFromAnotherScript("system", "chr_status_set_quartz_slot_level", [INT(0), INT(1), INT(9), INT(12)])
CallFunctionFromAnotherScript("system", "chr_status_set_quartz_slot_level", [INT(0), INT(1), INT(10), INT(12)])
CallFunctionFromAnotherScript("system", "chr_status_set_quartz_slot_level", [INT(0), INT(1), INT(12), INT(12)])
CallFunctionFromAnotherScript("system", "chr_status_set_quartz_slot_level", [INT(0), INT(1), INT(13), INT(12)])
CallFunctionFromAnotherScript("system", "chr_status_set_quartz_slot_level", [INT(0), INT(1), INT(14), INT(12)])
CallFunctionFromAnotherScript("system", "chr_status_set_quartz", [INT(1), INT(0), INT(3901), INT(12)])

```

By using `CallFunctionFromAnotherScript`, you can specify which script contains the function and the function name, as well as its parameters.

After selecting Choice0 then Choice1, the result is this:



We can see that Shizuna's level is 25, which likely comes from the `INT(25)` from the first call of `chr_status_set`.



We can also verify that she equipped the correct accessories, for example the second ring.



The last call to “chr_status_set_equip” has INT(1634) as the third parameter, which is the ID of the item to be equipped. If you decompile t_item.tbl with tbl2json.py, you will find the item with id = 1634 below:

```
{
  "id": 1634,
  "character_restriction": 65535,
  "text1": "",
  "text2": "DL",
  ...
  "stack_size": 99,
  "price": 9600,
  "animation": "",
  "name": "藍色徽章",
  "description": "【 <c698>ATS+20 90%防止「炎傷・黑暗・恐懼」</c> 】\n使用了蒼耀石，散發出清潔感的徽章。",
  "data": "C9 00 00 00 70 00 00 00 00 00 00 00 00 00 00 00"
```

which corresponds to what we can see on the screen.

Conclusion

So that was a “simple” example of a mod for Kuro no Kiseki that adds a character to the party and equips them with a few items. This hopefully should be a good enough basis for modders to start modding this game beyond the usual model swaps.

Below I will provide the full code for the TK_SHIZUNA function we made. It is still possible that this code could be improved though. Feel free to share your discoveries on the modding discord if you happen to find anything interesting or any issue that I have overlooked during this research. Also if you have any question, you can ask them on discord/on the github repository of KuroTools that was linked in the introduction.

The full TK_SHIZUNA function

```
set_current_function("TK_SHIZUNA")

CallFunction("TALK_BEGIN", [FLOAT(4.0), INT(0)])

CallFunction("menu_create", [INT(28), INT(0), INT(1), INT(1)])
CallFunction("menu_additem", [INT(0), "Add a character to the party", INT(1)])
CallFunction("menu_additem", [INT(1), "Add an item to the inventory", INT(1)])
CallFunction("menu_open", [INT(1), INT(-1), INT(-1), INT(1)])
CallFunction("menu_wait", [INT(1)])
AssignVar("VAR_0", LoadResult(0))
CallFunction("menu_close", [INT(1)])
CallFunction("menu_is_canceled", [INT(1)])
JumpWhenTrue("Loc_MenuCanceled", istru(LoadResult(0)))
SAVERESULT(0)
JumpWhenFalse("Loc_Choice0", equal(LoadResult(0), INT(0)))
JumpWhenFalse("Loc_Choice1", equal(LoadResult(0), INT(1)))
JUMP("Loc_ChoiceMade")

Label("Loc_Choice0")

Command("Cmd_text_06", ["Adding Shizuna to the party", INT(65534)])
```

```
CallFunction("wait_prompt", [])  
CallFunction("party_join", [INT(0), INT(-1), INT(12)])  
JUMP("Loc_ChoiceMade")
```

```
Label("Loc_Choice1")
```

```
Command("Cmd_text_06", ["Equiping stuff", INT(65534)])  
CallFunction("wait_prompt", [])  
CallFunctionFromAnotherScript("system", "chr_status_set", [INT(25), INT(0), INT(12)])  
CallFunctionFromAnotherScript("system", "chr_status_clear_craft", [INT(12)])  
CallFunctionFromAnotherScript("system", "chr_status_add_craft", [INT(0), INT(12)])  
CallFunctionFromAnotherScript("system", "chr_status_set_equip", [INT(1), INT(1), INT(601), INT(12)])  
CallFunctionFromAnotherScript("system", "chr_status_set_equip", [INT(1), INT(2), INT(704), INT(12)])  
CallFunctionFromAnotherScript("system", "chr_status_set_equip", [INT(1), INT(3), INT(1005), INT(12)])  
CallFunctionFromAnotherScript("system", "chr_status_set_equip", [INT(1), INT(4), INT(1205), INT(12)])  
CallFunctionFromAnotherScript("system", "chr_status_set_equip", [INT(1), INT(5), INT(1631), INT(12)])  
CallFunctionFromAnotherScript("system", "chr_status_set_equip", [INT(1), INT(6), INT(1634), INT(12)])  
CallFunctionFromAnotherScript("system", "chr_status_set_hollowcore", [INT(1), INT(3348), INT(12)])  
CallFunctionFromAnotherScript("system", "item_set_hollow_core_level", [INT(0), INT(2), INT(3348)])  
CallFunctionFromAnotherScript("system", "chr_status_set_artsdriver", [INT(1), INT(3450), INT(12)])  
CallFunctionFromAnotherScript("system", "chr_status_set_quartz_slot_level", [INT(0), INT(1), INT(0), INT(12)])  
CallFunctionFromAnotherScript("system", "chr_status_set_quartz_slot_level", [INT(0), INT(1), INT(1), INT(12)])  
CallFunctionFromAnotherScript("system", "chr_status_set_quartz_slot_level", [INT(0), INT(1), INT(2), INT(12)])  
CallFunctionFromAnotherScript("system", "chr_status_set_quartz_slot_level", [INT(0), INT(1), INT(4), INT(12)])  
CallFunctionFromAnotherScript("system", "chr_status_set_quartz_slot_level", [INT(0), INT(1), INT(5), INT(12)])  
CallFunctionFromAnotherScript("system", "chr_status_set_quartz_slot_level", [INT(0), INT(1), INT(6), INT(12)])  
CallFunctionFromAnotherScript("system", "chr_status_set_quartz_slot_level", [INT(0), INT(1), INT(8), INT(12)])  
CallFunctionFromAnotherScript("system", "chr_status_set_quartz_slot_level", [INT(0), INT(1), INT(9), INT(12)])  
CallFunctionFromAnotherScript("system", "chr_status_set_quartz_slot_level", [INT(0), INT(1), INT(10), INT(12)])  
CallFunctionFromAnotherScript("system", "chr_status_set_quartz_slot_level", [INT(0), INT(1), INT(12), INT(12)])  
CallFunctionFromAnotherScript("system", "chr_status_set_quartz_slot_level", [INT(0), INT(1), INT(13), INT(12)])  
CallFunctionFromAnotherScript("system", "chr_status_set_quartz_slot_level", [INT(0), INT(1), INT(14), INT(12)])  
CallFunctionFromAnotherScript("system", "chr_status_set_quartz", [INT(1), INT(0), INT(3901), INT(12)])  
CallFunctionFromAnotherScript("system", "chr_status_set_quartz", [INT(1), INT(1), INT(4320), INT(12)])  
CallFunctionFromAnotherScript("system", "chr_status_set_quartz", [INT(1), INT(2), INT(4020), INT(12)])  
CallFunctionFromAnotherScript("system", "chr_status_set_quartz", [INT(1), INT(4), INT(3911), INT(12)])  
CallFunctionFromAnotherScript("system", "chr_status_set_quartz", [INT(1), INT(5), INT(4011), INT(12)])  
CallFunctionFromAnotherScript("system", "chr_status_set_quartz", [INT(1), INT(6), INT(4000), INT(12)])  
CallFunctionFromAnotherScript("system", "chr_status_set_quartz", [INT(1), INT(8), INT(4101), INT(12)])  
CallFunctionFromAnotherScript("system", "chr_status_set_quartz", [INT(1), INT(9), INT(3701), INT(12)])  
CallFunctionFromAnotherScript("system", "chr_status_set_quartz", [INT(1), INT(10), INT(4310), INT(12)])  
CallFunctionFromAnotherScript("system", "chr_status_set_quartz", [INT(1), INT(12), INT(4220), INT(12)])  
CallFunctionFromAnotherScript("system", "chr_status_set_quartz", [INT(1), INT(13), INT(4211), INT(12)])  
CallFunctionFromAnotherScript("system", "chr_status_set_quartz", [INT(1), INT(14), INT(4120), INT(12)])  
CallFunctionFromAnotherScript("system", "item_set_artsdriver_slot_all_unlock", [INT(3450)])  
CallFunctionFromAnotherScript("system", "item_set_arts_plugin", [INT(1), INT(3543), INT(0), INT(3450)])  
CallFunctionFromAnotherScript("system", "item_set_arts_plugin", [INT(1), INT(3611), INT(1), INT(3450)])  
CallFunctionFromAnotherScript("system", "chr_status_set", [INT(0), INT(5), INT(12)])
```

```
JUMP("Loc_ChoiceMade")
```

```
Label("Loc_MenuCanceled")  
Command("Cmd_text_06", ["Menu Canceled", INT(65534)])  
CallFunction("wait_prompt", [])
```

```
Label("Loc_ChoiceMade")
```

```
CallFunction("TALK_END", [])
```

```
AssignVar("VAR_0", UNDEF(0x0))
```

```
SAVERESULT(0)
```

```
Return()
```